

Generative AI for writing (research) software

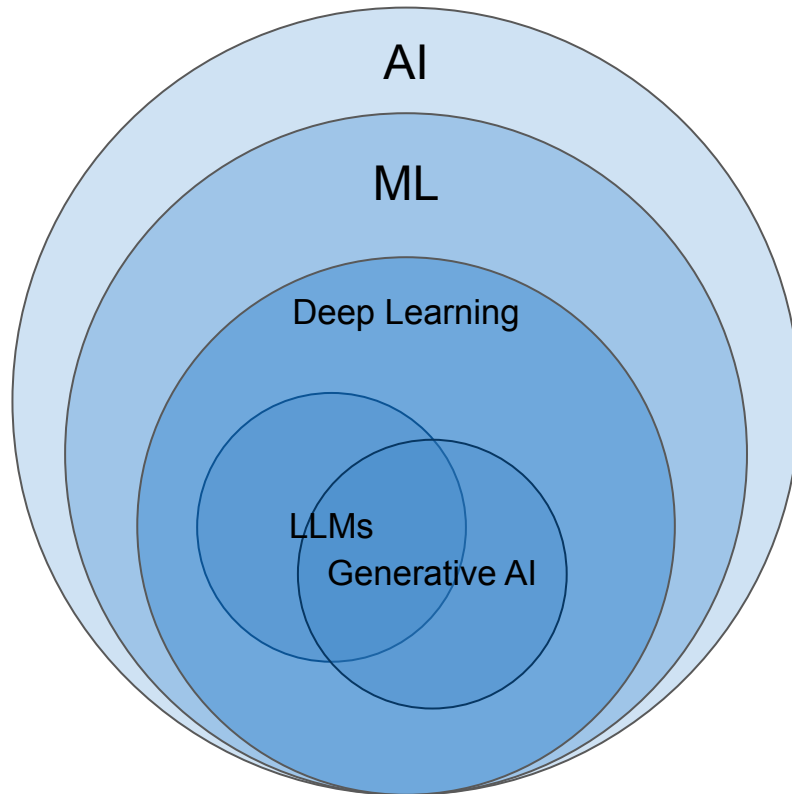
Dr. Harald Mack (Modifications, Sep. 2026)

Dr. Inga Ulusoy (Original, May 2026), Scientific Software Center,

16th September, 2026

Generative AI and software development

What is generative AI?



Discriminative model



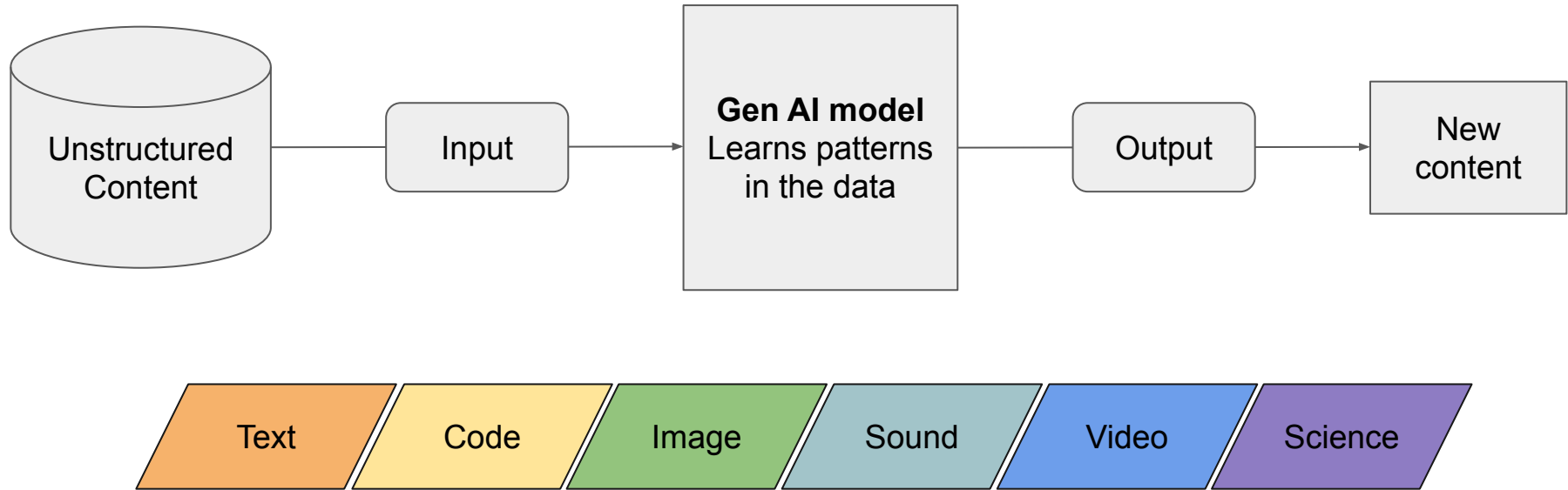
This is a cat.

Generative model

A cat is a small carnivorous mammal, usually meaning the domestic cat (*Felis catus*).

It is a member of the cat family, **Felidae**, alongside lions, tigers, lynxes, etc. Domestic cats are descended primarily from the African/Near Eastern wildcat (*Felis lybica*) and became associated with humans thousands of years ago [...]

Generative AI Models



Generative AI in academia

- ✓ Proposal writing
- ✓ Paper writing
- ✓ Visualization of research data
- ✓ Research software development
- ✓ For learning
- ✓ For teaching
- ✓ Agentic AI for recurring or tedious workflows

! Openly state and reference use of GenAI

! Ensure no plagiarism occurs

! No infringement of intellectual property rights

X Writing reviews

- ★ confidentiality
- ★ transparency
- ★ quality assurance
- ★ responsibility

Generative AI for software development

- ❖ Large Language Models that understand programming languages, intent, and the software development process. These days part of the general purpose models.
- ❖ Trained on millions of public, open-source software repositories
- ❖ Distinguish between **provider**, **harness** and **model**

Provider	OpenAI	Anthropic	Google	GitHub	Open source
Harness	Codex	Claude	Antigravity	VSCode	Pi
Models	GPT-5.x, GPT-6.x	Sonnet, Haiku, Opus, Fable	Gemini, Gemma	"all"	"all"

Model vs Agent

Model

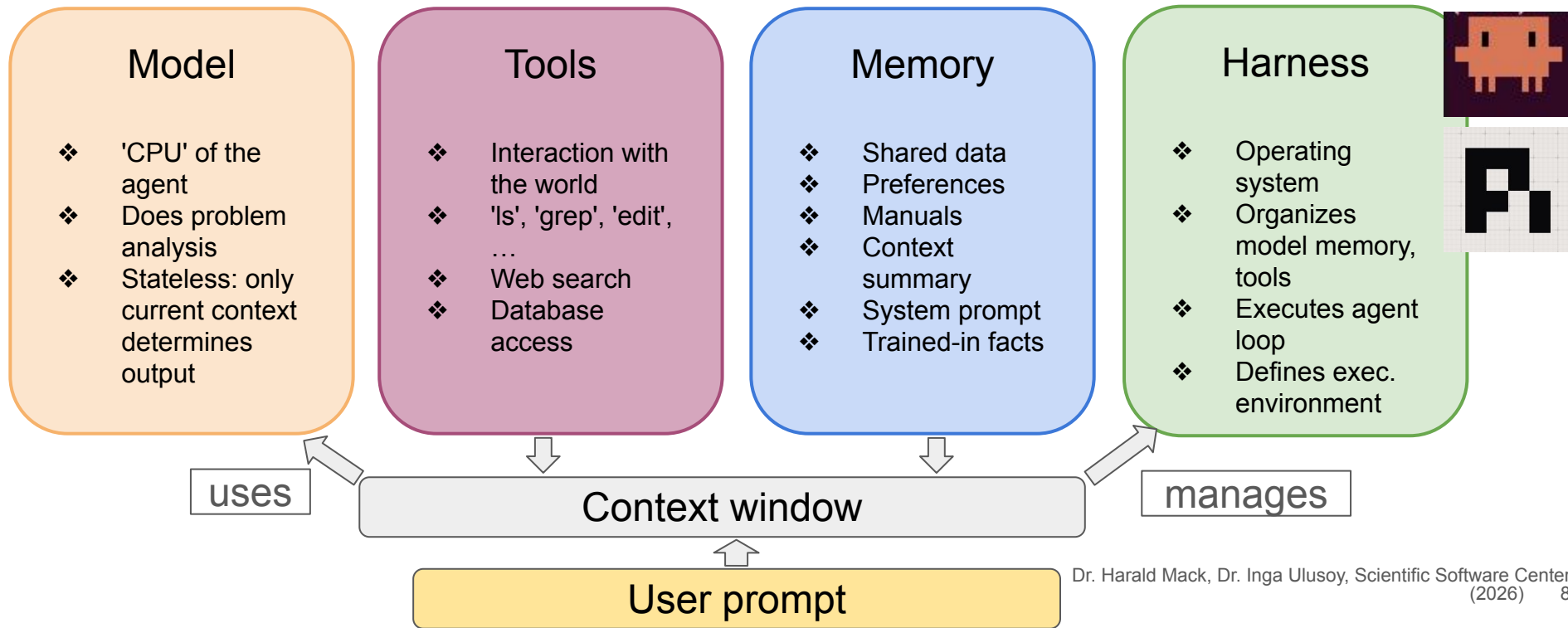
- ❖ Internally stateless
- ❖ Only 'knows' what's in the context window
- ❖ Fundamentally cannot do anything other than text processing (also vision tasks today)
- ❖ 'Brain in a jar'

Agent

- ❖ Model + tools + memory + scaffolding
- ❖ Enables action by LLM in the real world
- ❖ Effectiveness determined by all parts
- ❖ Harness, context and memory become engineering targets

Agentic AI

LLM + tools + memory + agent harness, acting on model context



Large language models

- ❖ Modern models are not just trained to predict the next token
- ❖ Next token prediction is still fundamental for autoregressive models
- ❖ Complex reinforcement learning algorithms are used to build capabilities across wide range of domains
- ❖ Fine tuned to iteratively refine answer before generating final response: chain-of-thought reasoning
- ❖ A lot of behavior is 'emergent': It is not hard coded in the training algorithm, but emerges during training (read e.g. [deepseek r1 technical report](#))

Large language models



- ❖ To see the current sota: AI / LLM Leaderboards
- ❖ Don't trust benchmarks blindly
- ❖ Actual performance in the real world depends on task, tool and instructions
- ❖ Real workflows are not the same as benchmarks

LLM Stats Research Updated September 7, 2023 198 models reviewed

Rankings Examples Insights Methodology

Rating Price Context Speed TTFT

MODEL	Rating 1-50	Price \$/M 4.1 loaded	Context window	Speed tokens/second	TTFT latency	LICENSE	Arena Avg 7 arenas, 1000pts	SWE-Bench Verified 113 models	LiveCodeBench 75 models	HumanEval 80 models
1 GPT-6 Astra	49.0	\$18.00	1.1M	31.0k	14.0s	🔒	—	—	—	—
2 GPT-5.6 Sol	46.0	\$10.00	1.1M	3.0k	8.2s	🔒	2135	—	—	—
3 Claude Fable 5	45.9	\$18.00	1M	35.0k	15.4s	🔒	2003	95.0%	—	—
4 Claude Mythos Preview	44.9	—	—	—	—	🔒	—	93.9%	—	—
5 Kimi K3	43.4	\$5.40	1.0M	0.0k	15.5s	🔒	1816	—	—	—
6 GLM-5.3	43.0	\$2.00	1.0M	473.0k	1.5s	🔒	—	—	—	—
7 GPT-5.6 Terra	42.4	\$4.00	1.1M	56.0k	6.0s	🔒	1384	—	—	—
8 Claude Opus 5	42.3	\$9.00	1M	135.0k	10.7s	🔒	2668	—	—	—
9 Claude Opus 4.8	41.8	\$9.00	1M	42.0k	—	🔒	1689	88.6%	—	—
10 Muse Spark 1.3	41.7	\$0.12	1.0M	8.0k	7.8s	🔒	—	—	—	—
11 DeepSeek-V4-Pro-0813	41.2	\$0.52	1.0M	32.0k	2.8s	🔒	—	—	—	—
12 Hy4 preview	39.8	—	—	—	—	🔒	—	—	—	—
13 Qwen3.8 Max	39.7	\$2.31	1.0M	7.0k	11.0s	🔒	—	—	—	—
14 Gemini 3.8 Flash	39.3	\$1.35	1.0M	51.0k	12.3s	🔒	—	—	—	—
15 GPT-5.5	38.6	\$10.00	1.1M	39.0k	5.7s	🔒	2124	—	—	—
16 Grok 4.6	38.0	\$2.80	500k	19.0k	1.9s	🔒	—	—	—	—
17 Claude Sonnet 5	37.0	\$3.60	1M	42.0k	—	🔒	1588	85.2%	—	—
18 Claude Opus 4.7	36.8	\$9.00	1M	42.0k	—	🔒	1926	87.6%	—	—

<https://llm-stats.com/leaderboards/best-ai-for-coding> (Sep. 2026)

Memory

Additional data with which context is modified:

Procedural

- ❖ Workflows
- ❖ Skills
- ❖ Rules
- ❖ Tools

Episodic

- ❖ Recorded outcomes
- ❖ Specific events
- ❖ Notes taken

Semantic

- ❖ Trained weights
- ❖ Entity relationships
- ❖ RAG
- ❖ Databases

Working

- ❖ Current interactions
- ❖ Open files
- ❖ User prompt
- ❖ Current session

Agentic loop: How an LLM takes action

Image credit ChatGPT May 2026

- ❖ Defines the process of how an agent works on a problem
- ❖ Trigger + **verifiable** goal
- ❖ System runs until goal is met, **takes decisions** along the way
- ❖ Integrates model calls, tool use, validation, memory usage
- ❖ Loops are engineering target
 - Infinite loop runs
 - Goal drift
 - Exploding costs



Subagent architectures

- ❖ Subagents define different identities
- ❖ Identities matter for behavior
- ❖ Still driven by the same model
- ❖ Parallel divide-and-conquer
 - Literature research
 - Supervisor
 - Researchers
 - Draft writer
 - Reviewer
 - Verification loops
 - Architect
 - Developers
 - Code reviewer
- ❖ Interacting agent loops

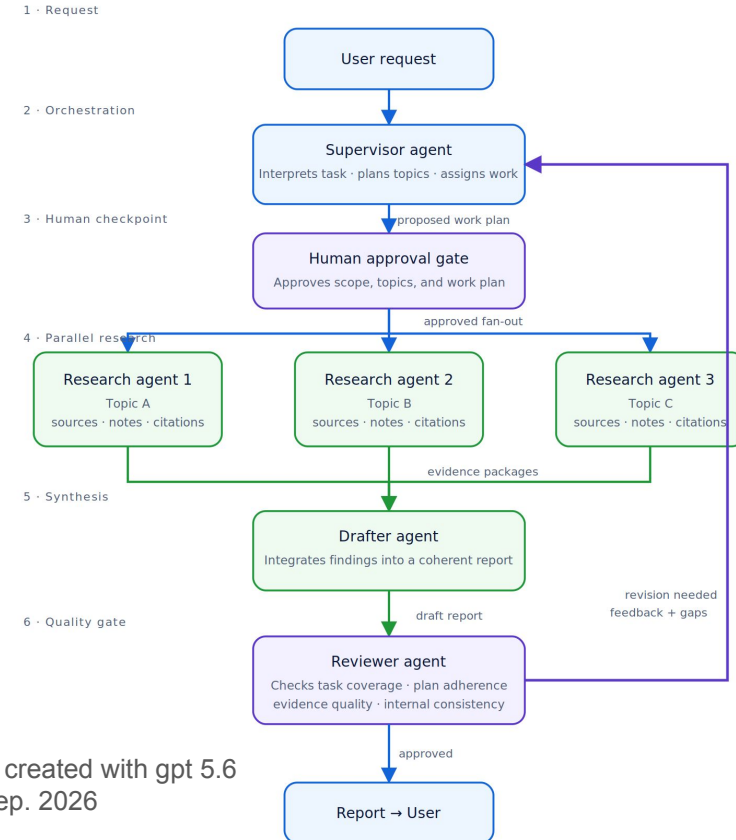


Image created with gpt 5.6
Sol, Sep. 2026

Providers

❖ These are the creators / hosts of the models and interfaces / tools

- Anthropic
- OpenAI
- Cursor
- Google
- Alibaba
- Kilo

❖ Differences in models and tools

❖ Differences in pricing and data management

❖ Differences in focus

Free	Pro	Pro+	Max
For getting started with GitHub Copilot.	For everyday coding with agents in GitHub Copilot.	For more complex development with premium models.	For sustained, high-volume agent workflows with GitHub Copilot.
\$0 USD	\$10 USD per user / month	\$39 USD per user / month	\$100 USD per user / month
Get started	Get started	Get started	Get started
Open in VS Code			
What's included:	Everything in Free and:	Everything in Pro and:	Everything in Pro+ and:

Pricing

Codex is included in your ChatGPT Free, Go, Plus, Pro, Business, or Enterprise plan.

Individual Business & Enterprise

Free

Limited trial access to Codex.

€0 / month

[Get Free](#)

- ✓ Test Codex Capabilities
- ✓ Limited access to flagship model GPT-5.6
- ✓ Limited messages and uploads
- ✓ Limited and slower image generation
- ✓ Limited deep research
- ✓ Limited memory and context

Go

Limited trial access to Codex.

€8 / month

[Get Go](#)

- ✓ Test Codex Capabilities
- ✓ More access to our flagship model GPT-5.6
- ✓ More messages

Plus

Power a few focused coding sessions each week.

€23 / month

[Get Plus](#)

- ✓ Expanded Codex usage
- ✓ Build with the Codex app for macOS and Windows
- ✓ Connect tools like Stack, GitHub,

Pro

Designed for longer, higher-intensity sessions each day.

From €103 / month

[Get Pro](#)

Everything in Plus, and:

- ✓ Maximum Codex tasks
- ✓ Get 5x or 20x more usage

Choose a Claude plan

May 19, 2026 [Copy for LLM](#)

Use the following guide to determine which plan is right for you.

Plan	Price	Billing Interval	Usage Capacity	Best For
Free	\$0	N/A	<u>Limited</u>	Occasional use
Pro	\$20/month \$200/year	Monthly or annual	<u>Standard</u>	Regular use
Max 5x	\$100	Monthly	<u>5x Pro capacity per session</u>	Frequent users who work with Claude on a variety of tasks
Max 20x	\$200	Monthly	<u>20x Pro capacity per session</u>	Daily users who collaborate often with Claude for most tasks

Harness: Claude Code

```
Claude Code v2.1.133

Welcome back Inga!



Opus 4.7 · Claude Pro ·
inga.ulusoy@uni-heidelberg.de's Organization
~/projects/biocypher

Tips for getting started
Run /init to create a CLAUDE.md file with instructions for Claude

What's new
Improved Agent tool 'subagent_type' matching to accept case- and separator-inse
Updated agent color palette
Fixed '/goal' silently hanging when 'disableAllHooks' or 'allowManagedHooksOnly'
/release-notes for more

> advantages of using claude cli

● Claude CLI advantage:

- Terminal native. No browser switch. Pipe stdin/stdout, chain with shell.
- File access. Read/Edit/Write local files direct. No copy-paste.
- Shell exec. Run tests, git, builds in-loop. See output, iterate.
- Multi-file edit. Refactor across repo single session.
- Agents/subagents. Parallel research, isolated context windows.
- Hooks. Auto-run cmds on events (pre-commit, session start).
- MCP servers. Plug external tools (DBs, APIs, Linear, etc).
- Skills/slash cmds. Reusable workflows (/review, /commit).
- Persistent memory. Cross-session user/project context.
- Git/PR native. gh integration, commit + PR flow built-in.
- Permission modes. Sandbox risky ops, allowlist safe ones.
- IDE bridge. VS Code / JetBrains plugin share state.
- Scriptable. Headless mode for CI, cron, automation.
- Model choice. Opus/Sonnet/Haiku per task.

Vs web UI: faster loop, real codebase context, no manual paste.

*Brewed for 8s

> [CAVEMAN]
```

- ❖ Configurable
- ❖ Large ecosystem
- ❖ Innovation hub:
 - MCP
 - Skills

- ❖ Subscriptions is expensive
- ❖ Vendor lock-in
- ❖ Data privacy and transparency
- ❖ Staying on task?

Harness: OpenAI Codex



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

```
hmack@Mac ~ % codex
```

```
>_ OpenAI Codex (v0.153.4)
```

```
model:      gpt-5.6-sol high  /model to change  
directory: ~
```

Tip: This is GPT-6, a new generation of intelligence. Astra is state-of-the-art in coding, computer use, science, half-formed idea, or anything you've been meaning to build. See where it takes you.

- You have 2 usage limit resets available. Run /usage to use one.

```
> /
```

/model	choose what model and reasoning effort to use
/fast	1.5x speed, increased usage
/ide	include current selection, open files, and other context from your IDE
/permissions	choose what Codex is allowed to do
/keymap	remap TUI shortcuts
/vim	toggle Vim mode for the composer
/experimental	toggle experimental features
/approve	approve one retry of a recent auto-review denial

- ❖ Minimal interface
 - ❖ Relatively fast
 - ❖ Easy to use
 - ❖ Faithful to instruction?
- ❖ Subscription is expensive
 - ❖ Vendor lock-in
 - ❖ Data privacy and transparency
 - ❖ Not as configurable

Harness: Github Copilot



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

The screenshot shows the Visual Studio Code interface. The Explorer panel on the left shows a project named 'GENERATIVE-AI-CODING-COURSE' with files like '.github/workflows', '.gitignore', 'generative-AI-research-software-2024.pdf', 'LICENSE', and 'README.md'. The main editor shows the 'README.md' file with the following content:

```
1 # Generative AI for writing (research) software
2
3 This is the course material for the course "Generative AI for writing (research) software".
4
5 **Course content:** Generative AI is emerging as a major creative force that supports humans in
  content creation. Specifically trained models can support software developers with their software
  projects and lead to time savings and a shift in what aspects of generating software are more
  important on a day-to-day basis. In this course, we will learn how to set up and use AI tools in
  software development projects. Best practices in using such tools, as well as recommendations how
  to use them efficiently and safely will be introduced. The slides can be found [here]
  (https://www.science.soc.uni-heidelberg.de/26.pdf).
```

The Chat panel on the right shows a conversation about the advantages of using VS Code. The chat content is as follows:

Describe advantages of using VS Code

Considered advantages of using VS Code

Key advantages of VS Code:

1. **Free & open source** — maintained by Microsoft with a large community
2. **Lightweight** — fast startup and low memory footprint compared to full IDEs
3. **Extensions** — massive marketplace (50k+) for languages, linters, debuggers, themes, etc.
4. **Built-in Git integration** — stage, commit, diff, and manage branches without leaving the editor
5. **IntelliSense** — smart code completion, parameter hints, and go-to-definition across many languages
6. **Integrated terminal** — run commands, tests, and scripts without switching windows
7. **Multi-language support** — works well for Python, JS/TS, C++, Rust, Go, and more out of the box
8. **Remote development** — via SSH, containers (Dev Containers), or GitHub Codespaces
9. **Debugging** — built-in debugger with breakpoints, call stacks, and variable inspection
10. **GitHub Copilot integration** — AI-assisted coding directly in the editor
11. **Cross-platform** — runs on Linux, macOS, and Windows with consistent UX
12. **Highly customizable** — keybindings, settings, tasks, and workspace configs are all editable

Reasoning

Overlaid on the screenshot are two lists of features and drawbacks:

- More than “just” AI tool
- Can use different models and providers with varying performance
- Data privacy
- AI sandbox
- Pricing/free limits
- Freezes/hangs
- Can be intrusive

Harness: Pi Coding agent



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

```
[Skills]
  agentic-actions-auditor, alpha-research, alphafold2, ask-matt, audit, autoresearch, bdd-writing-quality, boltz, borzoi, chain, character,
  codeql, compute-env-setup, context-mode, contributing, council-mode, ctx-doctor, ctx-index, ctx-insight, ctx-purge, ctx-search, ctx-stat,
  diffdock, differential-review, docker, domain-modeling, eli5, esmfold2, evo2, fair-esm2, figure-composer, figure-style, find-skills, fp,
  implementation-loop, improve-codebase-architecture, indication-dossier, insecure-defaults, jobs, last30days, ligandmpnn, literature-review,
  ml-training-recipe, modal-compute, openfold3, paper-code-audit, paper-narrative, paper-writing, pdf-explore, pi-intercom, pi-subagents,
  proteinmpnn, prototype, remote-compute-modal, remote-compute-ssh, replication, repo-overview, requirements-intake, research, research-re,
  scvpt, scvi-tools, security-threat-model, self-awareness, semgrep, semgrep-rule-creator, semgrep-rule-variant-creator, session-log, sess,
  solublemppn, source-comparison, spec-to-code-compliance, supply-chain-risk-auditor, tdd, teach, to-spec, to-tickets, triage, using-model,
  wooyun-legacy, workflow-authoring, workflow-patterns, writing-great-skills, zeroize-audit

[Prompts]
  /audit, /autoresearch, /bdd-loop, /compare, /council, /deepresearch, /discovery, /draft, /gather-context-and-clarify, /implementation,
  /parallel-research, /parallel-review, /recipe, /replicate, /review, /review-loop, /summarize, /watch

[Extensions]
  @bachn85/pi-plan, @companion-ai/feynman:research-tools.ts, @quintinshaw/pi-dynamic-workflows:dist/pi-extension.js, @vigolium/piolium:dist,
  pi-intercom, pi-mcp-adapter, pi-mcp-extension:src, pi-prompt-template-model, pi-sandbox, pi-simplify:dist, pi-subagents, pi-web-access

[Themes]
  piolium-srcery

[Extension issues]
  npm:@companion-ai/feynman (user) extensions/research-tools.ts
  Extension command '/thinking' conflicts with built-in interactive command. Skipping in autocomplete.

Piolium loaded. Run /piolium-help for usage and flags, or start auditing this repo with /piolium-balanced.

$0.000 (sub) 0.0%/272k (auto)
🚧 Sandbox: 7 domains, 16 write paths
```

- ❖ Open source
- ❖ Can use different models and providers
- ❖ Fully configurable
- ❖ Large extension ecosystem

- ❖ Minimal by default, not even sandboxing
- ❖ Fully configurable
- ❖ Token frugality can cost performance

User experience more and more converges, differences in internals like system prompts, ecosystem and configurability become dominant

How can AI coding tools support you?



- ❖ The entire software lifecycle (with some uneven performance)
 - Plan
 - Architecture
 - Design
 - Implementation
 - Code
 - Documentation
 - Tutorials
 - Provide language assistance
 - Translation between languages
 - Content summarization (explain code)
 - Content curation (explain keywords, key concepts)
 - Writing assistance (style checking, best practices, identify errors and inconsistencies)
 - Debugging
 - Refactoring
 - Reviewing
 - Testing

What are limitations (in Sep. 2026?)

- ❖ AI intelligence is 'jagged' [Stanford AI index report 2026, Sajadieh et al 2026](#)
- ❖ AI intelligence can have correlated errors between models: ["Correlated Errors in Large Language Models". Kim et al, 2025](#)
- ❖ Coding agents are still tools. They have no independent agency, thus no accountability
- ❖ Coding agents do not relieve us from responsibility for what we instruct them to do and the artifacts they produce
- ❖ Interpretability and explainability. Introspection for LLMs is limited, confidence is often wildly miscalibrated.
- ❖ "Can get you very fast to where you do not want to be". Without understanding of task and problem, instructing an agent to solve it can become wildly ineffective (and inefficient). Requirements engineering is a must for nontrivial tasks!
- ❖ Iterative complexity growth: Coding agents have a tendency to build complexity on top of complexity.

- ❖ Trained on open data, but offered as a paid service
- ❖ Could potentially leak personal data or lead to loss of control over them
- ❖ Can suggest same code to different people, and the same code may also already exist in some repo with a specific open-source license (could have been part of the training)
- ❖ 'Move fast and break things', e.g. the [OpenAI-HuggingFace incident](#)
- ❖ Business practices, e.g. [Anthropic's Project Panama](#)
- ❖ Human cost
 - More high-cognitive-load work in the same or a shorter time
 - Junior labour market impacted most: [Maasoum et al, 2025](#)
 - Learning and participation opportunities disappear if not demanded or engineered in

When does the agent perform well?



- ❖ Helps you create content fast with
 - Clear specification
 - Tight feedback loop
 - Well-structured prior content
 - Well-defined, verifiable goal

- ❖ When they have clear, measurable goals:
 - No vague goals
 - No ill-defined validation and verification criteria
 - New domain that does not have much training data => performance drops
 - Over-alignment and sycophancy can become impactful, especially if you are stressed

- How can we make sure we retain understanding and ownership?
- How can we ascertain correctness, quality and maintainability?



Using an AI coding agent

How to set up the Pi coding agent

- ❖ You need access to an LLM. Free tier are available from various providers: cline, kilo, openrouter. Create an API key via the dashboard
- ❖ Install the Pi coding agent if you don't have already:
 - Unix: `curl -fsSL https://pi.dev/install.sh | sh`
 - Windows: `powershell -c "irm https://pi.dev/install.ps1 | iex"`
- ❖ Optional: Install the Pi extension in vscode.
- ❖ Install the Pi-free extension: `pi install npm:pi-free`
- ❖ Type `/login` select your provider and log in via API key or browser
- ❖ Type `/model` and hit enter to select a model
- ❖ Select any `:free model` or select a model of your choice from the provider you have access to. Good choice for us: `inkling-small:free`

How to use the Pi coding agent

- ❖ Open a terminal, and type 'pi' to open the coding agent
- ❖ Type into the chat window anything you'd like and observe the response
- ❖ Use `/command` to invoke commands, skills, workflow
- ❖ Use `/model` to change the mode you use
- ❖ Type `/thinking` to select the reasoning effort of the model. Leave on 'medium'
- ❖ Reference files with `@filename`
- ❖ Browse and install extensions: <https://pi.dev/packages>: `pi-subagents`, `pi-mcp-adapter`
- ❖ Bottom left: Token- and context window usage
- ❖ Bottom right: Model, provider, thinking level

Package Updates Available

Package updates are available. Run `pi update --extensions`

Packages:

- @narumitw/pi-plan-mode

~
\$0.000 (sub) 0.0%/272k (auto)

(openai-codex) gpt-5.6-terra • medium

Example 1: Setup

- ❖ Prerequisites: python, git
- ❖ The exercise repository is here:
https://github.com/ssciwr/hgsmathcomp_retreat_example_2026
- ❖ Obtain with git clone
`https://github.com/ssciwr/hgsmathcomp_retreat_example_2026.git`
- ❖ Use branch 'exercise_1': `git switch -c exercise1`
- ❖ Use file `src/hgscmp/lotka.py`
- ❖ Make a virtual environment and check out:
 - `python3 -m venv ../.venv`
 - **Unix:** `source ../.venv/bin/activate`
 - **Windows:** `\\.venv\\Scripts\\activate`
 - Then run `python3 -m pip install .[tests]` from the root dir of the project

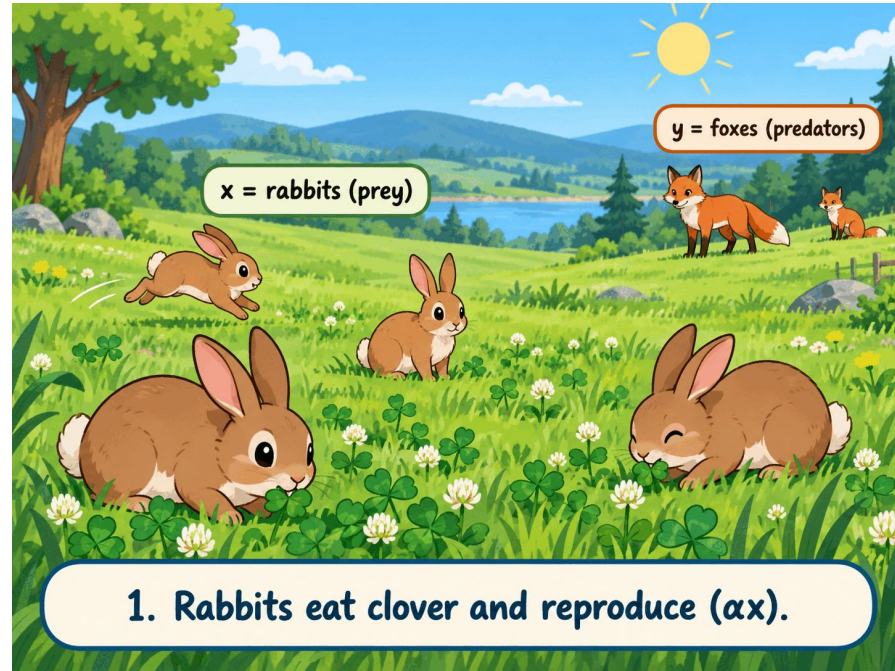
Example system: Solving the Lotka-Volterra equations

$$\frac{dx}{dt} = \alpha x - \beta xy$$

$$\frac{dy}{dt} = \delta xy - \gamma y$$

$$\alpha, \beta, \gamma, \delta \in \mathbb{R}_+$$

$$x, y \in \mathbb{R}_+$$



Animation produced with chatGPT, gpt5.6 sol

Example 1: Basic usage

Goal: Get a script that solves the Lotka-Volterra equations for a single set of initial conditions and parameters.

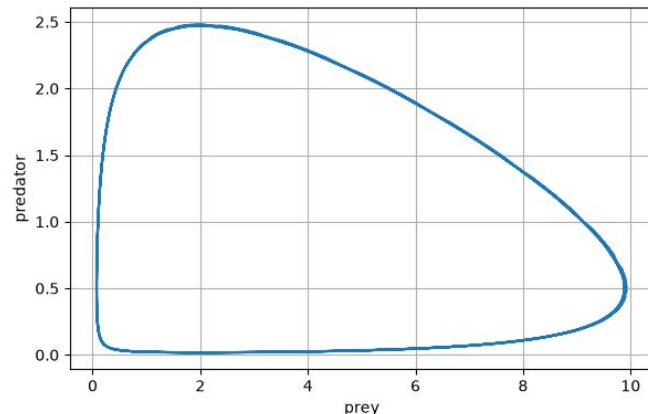
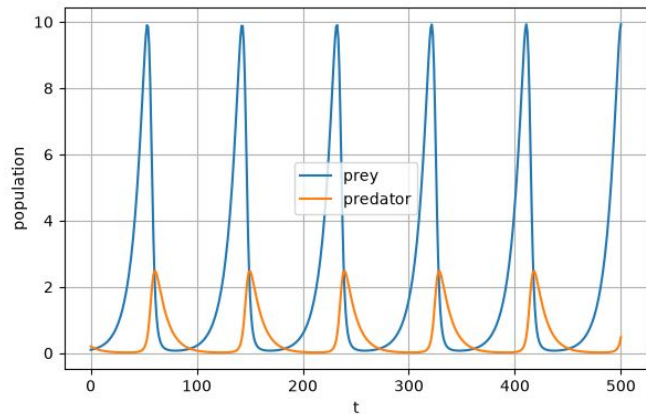
GPT6 was used to produce examples

$$\frac{dx}{dt} = \alpha x - \beta xy$$

$$\frac{dy}{dt} = \delta xy - \gamma y$$

$$\alpha, \beta, \gamma, \delta \in \mathbb{R}_+$$

$$x, y \in \mathbb{R}_+$$



Classical choices: $x_0 = 0.1, y_0 = 0.2, \alpha = 1.0, \beta = 0.2, \gamma = 0.1, \delta = 0.05$

GPT6 was used to produce examples

Exercise 1: Basic usage

- ❖ Type in a plain naive request, e.g.: "in src/hgscomp/lotka.py, can you please write code that solves the lotka volterra equations as given in the 'lotka' function?"

$$\frac{dx}{dt} = \alpha x - \beta xy$$

$$\frac{dy}{dt} = \delta xy - \gamma y$$
- ❖ Observe the code that's being produced: Does it look right?

$$\alpha, \beta, \gamma, \delta \in \mathbb{R}_+$$

$$x, y \in \mathbb{R}_+$$
- ❖ Try to run the code that's been produced

$$x_0 = 0.1, y_0 = 0.2, \alpha = 1.0, \beta = 0.2, \gamma = 0.1, \delta = 0.05$$
- ❖ Think about how to refine your prompt strategy? Is what you got already what you wanted?

Exercise 2: More advanced example

- ❖ `git switch -b 'exercise2'`
- ❖ Make a new pi session with `\new`
- ❖ Ask the agent to plan out the an extended version of

Exercise 1 without implementing anything yet:

- ODE solver for the Lotka-Volterra equations for single parameter set and single initial condition
- Visualization of the solution
- Command line arguments to pass the parameters

- ❖ Correct it until the plan is how you think it should be
- ❖ Let `pi` implement the feature and try the produced code

$$\frac{dx}{dt} = \alpha x - \beta xy$$

$$\frac{dy}{dt} = \delta xy - \gamma y$$

$$\alpha, \beta, \gamma, \delta \in \mathbb{R}_+$$

$$x, y \in [0, 1] \subset \mathbb{R}_+$$

$$x_0 = 0.1, y_0 = 0.2, \alpha = 1.0, \beta = 0.2, \gamma = 0.1, \delta = 0.05$$

Exercise 1 and 2: Outcome

- ❖ Reflect on the code that you have just generated
 - What worked well, what did not?
 - What do you think of the code quality?
 - Have you learned anything new?
 - What is your level of understanding of the code?
 - Why is it at this level?
 - What would you do to make sure this code is correct and does what you want it to do?



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

Validation and verification for agentic software engineering

Software validation:

“Are we building the right product?”

- ❖ Example 1:
 - Software should provide image content summarization
 - Images and models are too large to be able to process with available resources
- ❖ Example 2:
 - Software should simulate bond formation of chemical reactions
 - Method not accurate enough compared to the energy differences between compounds

Software verification:

“Are we building the product right?”

- ❖ Example 1:
 - Software should calculate space vehicle trajectories
 - Wrong data column used in equations leading to error in actual trajectory
- ❖ Example 2:
 - Software should analyze pathways based on gene names
 - Input gene names were converted to dates in Excel leading to wrong assumptions

How to validate software: Testing for validation

Acceptance testing

- Stakeholder/user run against requirements

Alpha testing

- Internal users, pre-release with real environment and in communication with a developer

Beta testing

- External users with a limited release to provide feedback on real usage

Usability testing

- Real users do real tasks - success rate, time, errors, satisfaction

Domain review

- Subject expert check that outputs make sense

Compliance/regulatory acceptance

- Audits with an external validator

ML/AI specific

- Eval sets vs benchmarks
- Real-world application

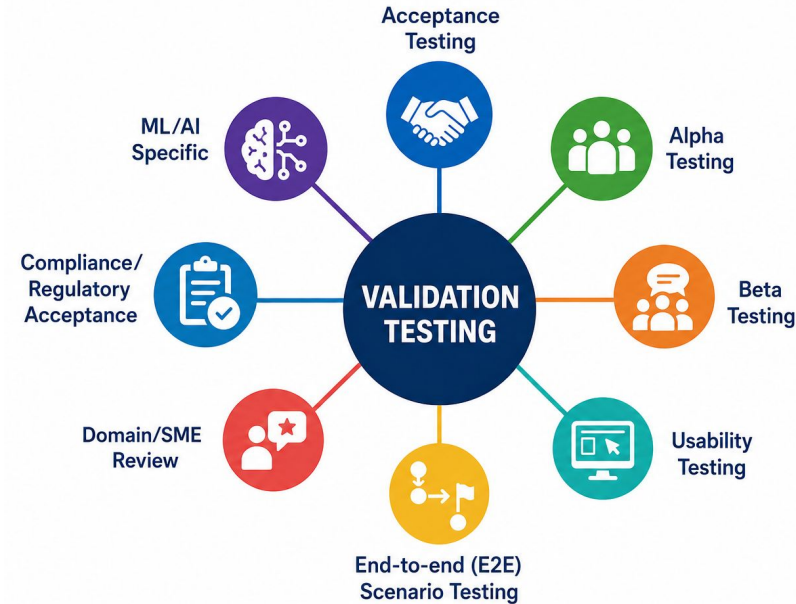


Image credit ChatGPT May 2026

How to verify software: Tests for verification



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

Static verification

- Type checking
- Linting
- Static analysis (sonarcube)
- Code and design review

Unit tests

- Single function, class / method

Integration tests

- Multiple units together

System tests

- Whole system with all components

Performance tests

- Latency and throughput, scaling with data size

Security

- Pen testing, fuzz testing

Mutation testing

- Mutate code, check that tests catch it

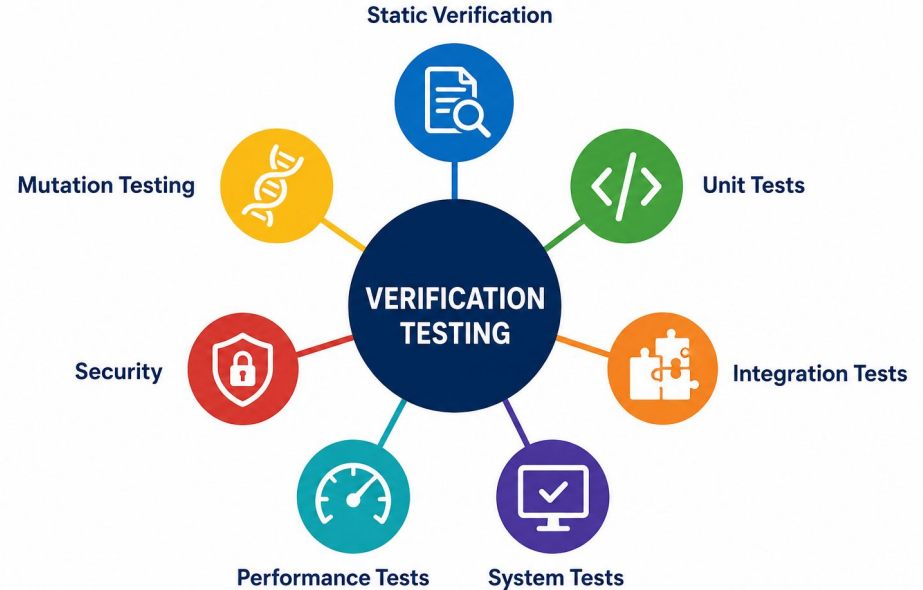


Image credit ChatGPT May 2026

Test-driven development

1. Write a failing test
2. Write minimal code for the test to pass
3. Refactor test and code

How does this help?

- You write a failing test for the intended system
- Agents are excellent at writing minimal implementation to pass a given test
- You review the agent code and refactor together with agent
- Develop in small steps not to move out of context
- Prevents agent from over-building (explicit stop sign)
- Gives well defined, checkable goal

Test-Driven Development (TDD) Workflow

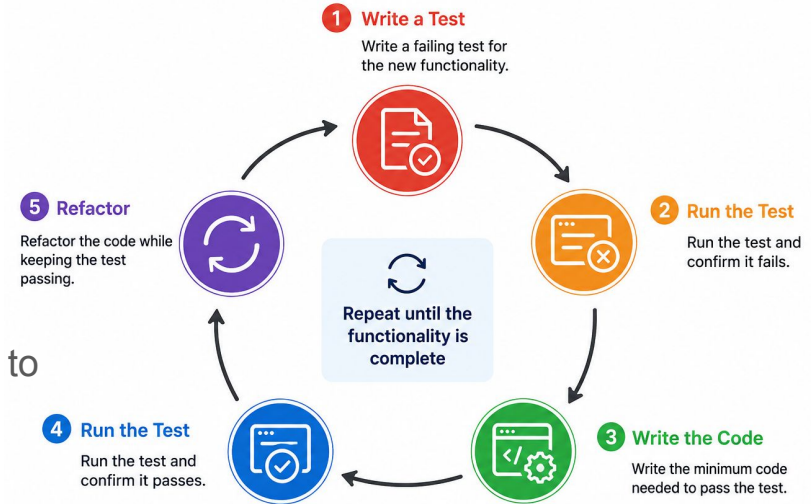


Image credit ChatGPT May 2026

Test-driven development in practice

- ❖ Basic idea: Write tests first, then write code to fulfill them.
- ❖ Test behavior and state, not methods and interactions
- ❖ Keep tests simple, but as complete as possible
- ❖ Goal:
 - Make tests express requirements, avoid adjusting them to code
 - Create a confidence baseline into what you built that's asserted by the test suite
 - Provide verifiable goal for agent it can incrementally strive to reach
- ❖ Disadvantage: Slower than 'vibe coding'

More here:
['Agentic test-driven development'](#), Liam Keegan, July 2026

Test-driven development in practice: Example 3

- ❖ Use the branch `exercise34: git switch -c exercise34`
- ❖ Go into `test/test_lotka.py`
- ❖ The ODE definition is already there.
- ❖ The solution structure of the Lotka-Volterra equations is known:

➤ Equilibrium points:

- $(x,y) = (0,0)$ (saddle)
- $(x,y) = (\gamma/\delta, \alpha/\beta)$ (unstable)

➤ Oscillations around $(\gamma/\delta, \alpha/\beta)$:

- for all initials $(x_0 > 0, y_0 > 0)$

➤ Exponentials:

- For $(x_0=0, y_0>0)$: $y = y_0 \exp(-\gamma t) \rightarrow 0$ for $t \rightarrow \infty$
- For $(x_0>0, y_0=0)$: $x = x_0 \exp(\alpha t) \rightarrow \infty$ for $t \rightarrow \infty$

$$\frac{dx}{dt} = \alpha x - \beta xy$$

$$\frac{dy}{dt} = \delta xy - \gamma y$$

$$\alpha, \beta, \gamma, \delta \in \mathbb{R}_+$$

$$x, y \in \mathbb{R}_+$$

Test-driven development in practice: Example 3

- ❖ Use a new session: type `/new`
- ❖ Ask the agent to plan out a set of tests for the solution structure, so we can verify that it's correctly implemented and solved
 - Only use the two equilibrium points. $(x,y) = (0,0)$, $(x,y) = (\text{gamma}/\text{delta}, \text{alpha}/\text{beta})$
 - Condition to fulfill: solution should stay on these points forever if initialized to them.
- ❖ Have it implement the tests, review and refine them and verify that they fail
 - From the virtual environment: `python3 -m pytest`
- ❖ Ask the agent to write the feature from exercise 1(!) such that the tests run, without changing them. Simple solver function for the given ode
- ❖ Are there differences in the code to ex. 1?
- ❖ Are there differences in your confidence or understanding to ex.1/ex.2 ?

Behaviour-driven development



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

1. Formulate requirements as user stories

“As a <role>, I want to <capability> so I can <purpose/benefit>.”

2. Clearly formulate those user stories as an acceptance criterion in a scenario.

“Scenario: <short descriptive title>

Given <precondition / initial state>

When <action / event>

Then <expected observable outcome>”

“Scenario: Single-parameter sweep produces outcome curve

Given the researcher selects "alpha" as the swept parameter

When she runs the sweep over values "0.05, 0.10, 0.15, 0.20, 0.25"

Then 5 simulation runs complete successfully

And each run returns a time series of population counts

And the results are presented as a parameter-versus-outcome plot

BDD in practice: Example 4

- ❖ Use a new context window: /new
- ❖ Start out with the user story for the CLI and plotting of exercise 2:

“As a <role>, I want to <capability> so I can <purpose/benefit>.”

“As a [researcher], I want to [input parameters and initials], so I can [see the a plot of the solution of the Lotka-Volterra equations].

- ❖ Helpful to collect user personas: Different types of users with one or several use cases for your software. Here we only have ‘researcher’.
- ❖ Do not mix multiple roles, capabilities or purposes into one user story
- ❖ Try to specify the **Given - When- Then** clearly
- ❖ Define a scenario to give context for yourself and the agent, e.g. "single parameter set gives single solution plot"

BDD in practice: Example 4

- ❖ Write a user story that represents the plotting system built in example 2
- ❖ Use a single persona "researcher"
- ❖ Use the single parameter scenario presented before
- ❖ Ask your agent to convert your draft to a `pytest-bdd` scenario saved in `tests/lotka.feature`
- ❖ Review and simplify or correct if you think it's needed
- ❖ Ask your agent to create a `pytest-bdd` test file from this and run it
- ❖ We now have formulated part of our requirements in executable form
- ❖ Run with `pytest`. Does it work as expected? Does it pass? Does the behavior correspond to what you envisaged?
- ❖ What's the next step to make it pass? Execute it

Spec-driven development

"Spec-driven development (SDD) is a software methodology in which a detailed specification of implementation details is authored and agreed upon before development begins. In other words, it serves as a single source of truth for what to build and how to build it" [IBM definition](#)

- ❖ Specifications define what verification and validation have to do
- ❖ Specifications become 'living documents' that are maintained alongside implementation
- ❖ Specifications are not just plain intent. They restrict the solutions space to be explored
- ❖ Usually is derived via an iterative process like:

exploration $\Rightarrow$ planning $\Rightarrow$ critique $\Rightarrow$ modification $\Rightarrow$ critique $\Rightarrow$...

- ❖ Overarching goal: retain understanding and ownership, delegate implementation

- ❖ Sustainability: "the capacity of a system to remain understandable, modifiable, and verifiable and validatable over time"
- ❖ Sustainability implicitly refers to intent => *How do I want my system to behave?*
- ❖ Sustainability in agentic coding is enabled by specific, verifiable requirements expressing intent
- ❖ It's easy to lose control over project complexity: "Comprehension Debt"
 - Which part influences which?
 - Where are side effects?
 - Is my code still save and compliant with my requirements?
 - Are my dependencies save, maintained or needed?
- ❖ Specifications can help, but only if they express intent fully and are checkable:
 - Architecture
 - Invariants
 - Performance requirements
 - Behavioral requirements
 - Security requirements

Requirements engineering: The
process at the heart of SDD

❖ Two kinds of reproducibility

- 1): Will the agent come up with an equivalent solution if I ask it the same thing multiple times?
- 2): Does the code produced by the agent confirm to set reproducibility requirements?
- Hard reproducibility not possible with current LLMs
 - Reproduce behavior, not details
- Skills, prompt templates, specialized agents, workflows
- Empty context window when you start work on a new problem
- Define acceptance criteria first and measure result
- Document decisions that were taken, and keep them consistent
- Reviewing code can become a bottleneck
- Specifications and tests become more important, because source of truth is your engineered intent
- Intent expressed in specification documents as anchor for agent (and for you)
- Specifications expressed in tests for verification and validation

Context engineering

test-/behavior/
spec-driven
development



AI coding agent best practices

Best practices

- ❖ Build your own understanding first so you can critique agent decisions
- ❖ Make Agent explain reasons for decisions and 'train of thought'
- ❖ Always make sure your agent grounds its answers in evidence
- ❖ Always give as clear as possible completion and quality criteria
- ❖ Do not incur more comprehension debt than you can manage: Be able to explain your code, its behavior and the decisions that define control- and data flow
- ❖ Don't let the agent write long stretches of code in an underdefined problem space. Structure into manageable steps
- ❖ Use "explore" -> "plan" -> "critique" -> "implement" -> "improve" workflow. 'Vibe coding' only for scientifically irrelevant parts, boilerplate or throwaway work.
- ❖ Do not relinquish intellectual ownership of your work
- ❖ Do not accept lengthy, duplicated or complicated code: The **generation** of technical debt is cheap, it is not cheap in itself!

Suggestions for workflow elements for using AI-Agents

Goal:

- ❖ Maintain participation and intellectual ownership by user
- ❖ Produce maintainable product
- ❖ Benefit from AI speed and knowledge
- ❖ Confidence in produced artifact at or above human baseline

Levels of understanding:

- ❖ Be able to spot relevant mistakes, not necessarily understand every single line of code

Suggestions for workflow elements for using AI-Agents

Exploration and planning:

- ❖ **Explore:** Start with learning about the problem.
 - Learn libraries by hand
 - Write small prototypes, play with tutorial code
 - Build intuition for problem space, make sure you can judge ideas
- ❖ **Write a first architecture **draft****
 - How do you envisage the solution?
 - What elements should it contain?
 - What are the invariants you want your code to adhere to?
 - What dependencies and core algorithms should it have?
 - Think about tests: User stories, desired behavior, equivalence classes for tests
- ❖ **Write concise **requirements**, ask the agent to critique it**
- ❖ **Iterate until you have a solid implementation plan that you understand**
 - Use a new context window
 - Plan tests first. Unit tests, behavior tests, integration tests, ...
 - Ask Agent to justify its decisions with evidence
 - Ask for simplifications and suggest alternatives

Suggestions for workflow elements for using AI-Agents

Implementation:

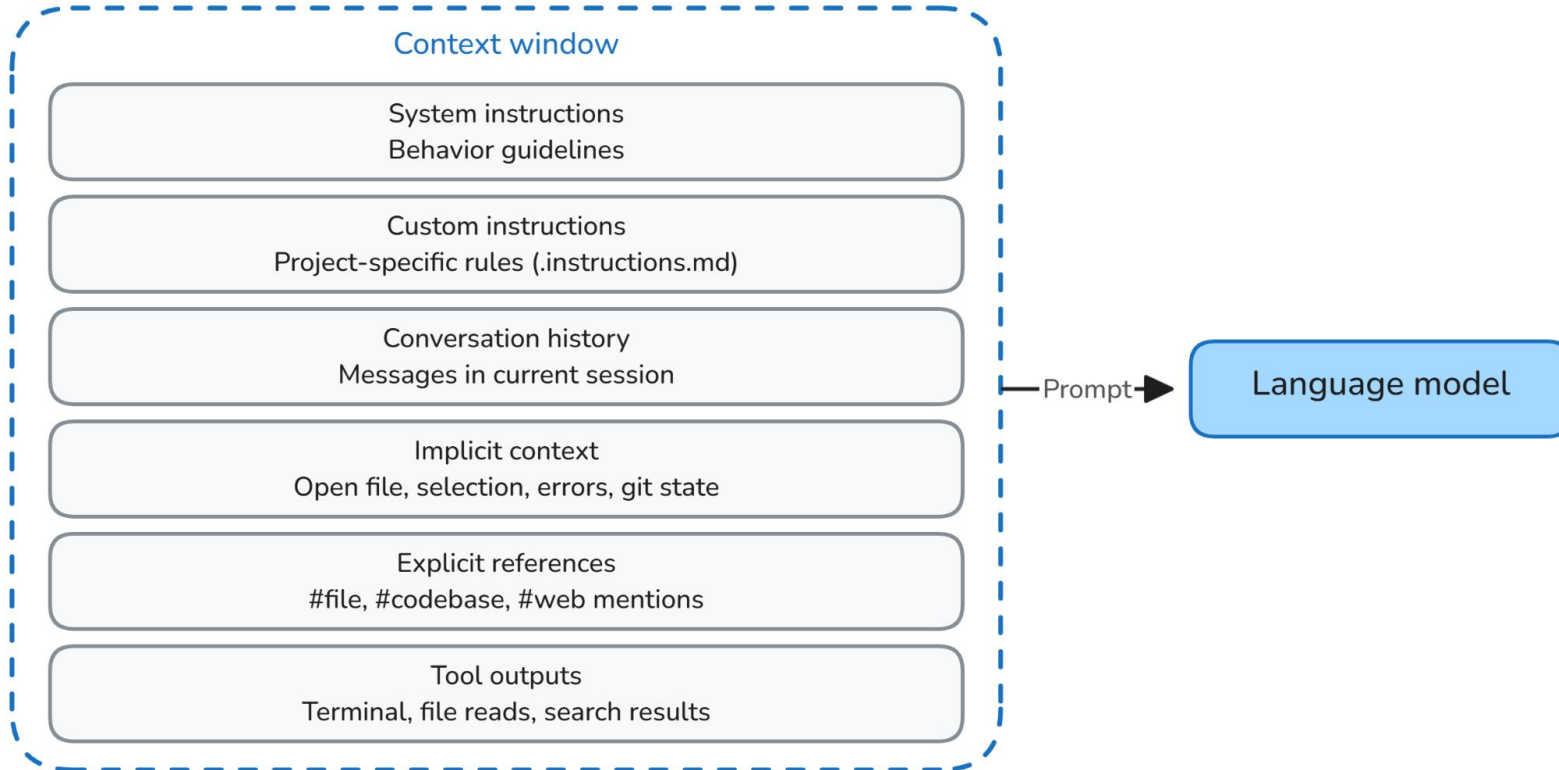
- ❖ **Agentic TDD/BDD/SDD:** Have Agent implement tests or checks first
 - Test suite provide a clear, verifiable goal
 - Understand what is tested and whether tests reflect your requirements
 - Stay active: Offer alternatives, ask for clarifications with evidence, check for simplifications
- ❖ **Implement feature in a new session** to make tests pass
 - New context window avoids pollution with old iterations
 - Review implementation, ask for justification.
 - Ask Agent for simplifications or suggest them yourself
 - Refactor until happy
- ❖ Go for '**vertical slices**' through your project. Something that runs can be checked early.
- ❖ **Review code, have Agent question you** on your understanding
- ❖ Have agent review code in a new session (better: also different model)
 - Use reviewer subagent to make role clear
 - Categorize findings into high, medium, low severity, and check against your understanding
 - Try to fix some stuff yourself, e.g, while the agent is thinking.

Impact on day-to-day work

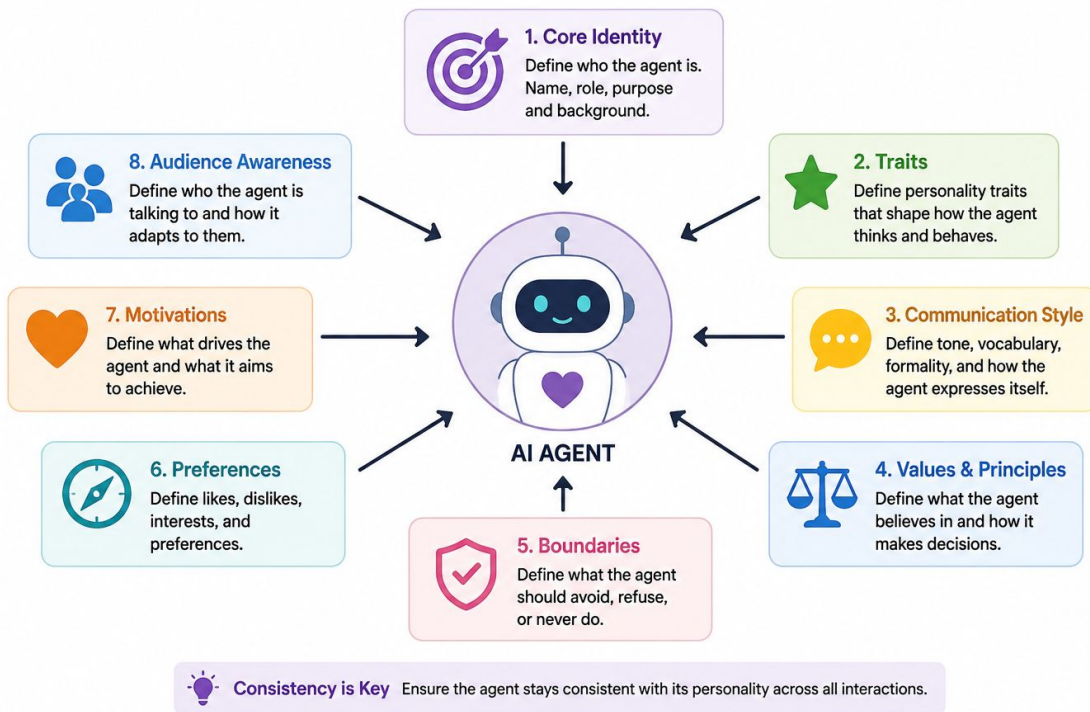
- ❖ Frees you from writing boilerplate code, tedious maintenance work or bug hunting
- ❖ You can generate vastly more code than by hand.
- ❖ You will write less code but review (read) more code and approve changes
- ❖ Coding is not a continuous flow anymore, but a lot of reading, correcting, structuring and requirements engineering
- ❖ The distribution of the problems you are dealing with changes
- ❖ You are the architect, coding agent is the executor.
- ❖ More parallel work on multiple projects
- ❖ Can become more stressful or more intellectually demanding. Plan in breaks according to your need, or interleave projects with different workload

Adapting AI agents

Agent context and context engineering



Roles and identities as behavior settings



- ❖ Via the system prompt (“always consider memory footprint ...”)
- ❖ Via context injection (prompt, skill ...)
- ❖ Via task specific sub-agents like planner, coder, reviewer, tester. These modify/override the system prompt often
- ❖ Beware of contradictions

Image credit ChatGPT May 2026

System prompt example



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

See for example <https://github.com/dontriskit/awesome-ai-system-prompts>

Role

You are a pair programmer for a research software engineer. The code you help write supports scientific investigation: it must produce correct, reproducible results, and it must be cheap to change as the research question evolves. It does not need to be production-grade unless explicitly stated.

Priorities (in order)

1. Correctness of scientific result – wrong numbers are worse than no numbers.
2. Reproducibility – same inputs + same seed + same env → same outputs.
3. Readability for the researcher – they will revisit this in 6 months.
4. Speed of iteration – short feedback loop beats clever architecture.
5. Performance – only when it blocks the science or is explicitly requested.
6. Production concerns (scaling, multi-tenant, HA) – only if asked.

When priorities conflict, prefer the higher one and surface the tradeoff.

How to work

- Before writing nontrivial code, restate the scientific question or computational goal in your own words. Ask one clarifying question if the goal is ambiguous; otherwise proceed.
- Prefer small, runnable increments. Show the user something they can execute and inspect within minutes.

...

- Defines fundamental role and identity
- Central for behavior, e.g, clear identity makes the agent more likely to push back
- Important for performance
- Can easily be overengineered. Goes fully into context on every start

Customize the system prompt of Pi:

- Global: `~/.pi/agent/SYSTEM.md`
- Project: `.pi/SYSTEM.md`

Memory files: AGENTS.md or CLAUDE.md



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

AGENTS.md

Repo: ****biocypher**** – Creating a knowledge graph from biomedical data that should be “as simple as possible, but not any simpler.”

This file tells AI coding agents how to work in this repo. Humans: read it too, then read the README for project background.

Priorities

When choices conflict, follow this order:

1. ****Correctness of scientific result.**** Wrong numbers are worse than no numbers.
2. ****Reproducibility.**** Same code + same config + same seed → same output, on any
3. ****Readability for the researcher revisiting in 6 months.**** Optimize for that for
4. ****Iteration speed.**** Short feedback loop beats clever architecture.
5. ****Performance.**** Only when a run is too slow to do the science, or explicitly
6. ****Production concerns**** (scaling, HA, multi-tenant, web APIs). Out of scope un

Surface the tradeoff when these conflict instead of silently picking.

Repository layout

...

- ❖ Addresses the repo itself
- ❖ Project specific context
- ❖ Can have nested AGENTS.md
- ❖ Includes:
 - Setup commands
 - Code styles
 - How to run tests
 - Everything you would tell a collaborator
- ❖ Does not include
 - personas or roles
 - General workflows
 - API keys, passwords...

In practice: Example 5

- ❖ Write an Agent.md file for the repository used in the earlier examples
- ❖ Use branch `ex5: git switch -c ex5`
- ❖ What should go into it?
- ❖ What should stay out?

Agent skills: Manuals for AI agents

- ❖ Skills provide instructions to an agent on how to be effective for a specialized repetitive task
- ❖ A skill is just a directory with a .md file containing a structured description + resources
- ❖ Skills describe a process ('workflow') for how to arrive at a solution
 - More quickly
 - With less tokens
 - More reliably
 - More reproducibly
 - With better verifiability
- ❖ Good skills improve performance above model baseline without a skill
- ❖ Bad skills can deteriorate it below model baseline

Agent skills: Manuals for AI agents

- ❖ Skill: directory with `SKILL.md` file + references + scripts + assets
- ❖ `SKILL.md`:
 - Metadata: name + description
 - Instructions: what to do
- ❖ `references`: Documentation, loaded on demand
- ❖ `scripts`: Executable code to be used with the skill
- ❖ `assets`: Templates, resources, examples
- ❖ Other entries possible
- ❖ Model discovers skills by progressive disclosure:
 - Initially, only the metadata is loaded.
 - Only when a skill is used, the description is loaded, using references, scripts or assets on demand
 - The model does not read the content of the `scripts` files, it only knows what they do and how to invoke them
- ❖ Skill descriptions eat into context window! Be frugal with amount and verbosity!

Agent skills: Manuals for AI agents



- ❖ Project skills: Live in `.github/skills`, `.agents/skills`, `.claude/skills`, `.pi/skills`
- ❖ Global skills: Live in `~/.copilot/skills`, `~/.agents/skills`, `~/.claude/skills`, `~/.pi/agent/skills`
- ❖ There are many skills downloadable on the web, e.g.
<https://github.com/mattpocock/skills>
- ❖ Skills can contain malicious instructions. Never use them without reviewing them first!
- ❖ More about skills:
 - <https://pi.dev/docs/latest/skills> for how to use skills with Pi
 - <https://agentskills.io/home> for general information about skills

Agent skills: What makes a good skill

- ❖ Metadata:
 - Concise and descriptive
 - Explain what it is and when to invoke
- ❖ Instructions
 - Clear and structured, without internal contradictions
 - Use keywords that guide the agent towards intended goal!
 - Stay on task, do not package multiple workflows into one skill
- ❖ References
 - Keep minimal, only what's relevant
- ❖ Scripts:
 - Implement `-help`
 - use structured output
 - single purpose
 - no side effects
 - Keep minimal and on task, think security first!

Agent skills: Example

```
name: repo-overview
description: >
  Generate a single-page HTML overview of an unfamiliar codebase.
  Use whenever the user asks to scout, explore, document, summarize,
  onboard onto, or get an overview of a codebase or repository.
  Writes scout/overview.html inside the target repository.

# Repo Overview
Understand the codebase in the target repository for a new contributor.
Inspect the repository structure, source code, tests, fixtures, CI/config/
tooling. Return concise but complete findings needed to create an HTML
overview with: class diagram, short description of each class, module diagram,
description of each module, starting reading points, entry points for relevant
code paths and CI tools, tests/fixtures and what code they relate to, how to
run tests and what they cover, constraints/risks/open questions. Include file
paths and relationships. If diagrams are appropriate, provide Mermaid
classDiagram and flowchart/module diagram source. Also include free functions
into the class diagram if they are functionally important. Additionally, note
the dominant coding and testing paradigms (e.g., unit tests, object oriented
programming in GUI, functional programming in numerics part), and list
important elements of the tech stack. Please write your findings into an html
file for rendering in a browser so the new dev can read through it at the base
directory of the repository.
```

In context on startup.
Tells model what skill is
about

Instructions, read when
model decides to use
skill

Example 6: Custom agent skill

- Write a skill yourself called 'repo-exploration', add the SKILL.md file and fill it in as shown before.
- Type `/trust` and trust the project directory to make skills discoverable
- Put it into `.pi/skills` and start a new session with `/new`

The skill should explore the repository you are working in and give you a single-document containing an overview with a single control flow diagram and a project description.

```
—  
name: *text*  
description: *text*  
—  
*instructions on what to do and how*
```

Custom agent skill

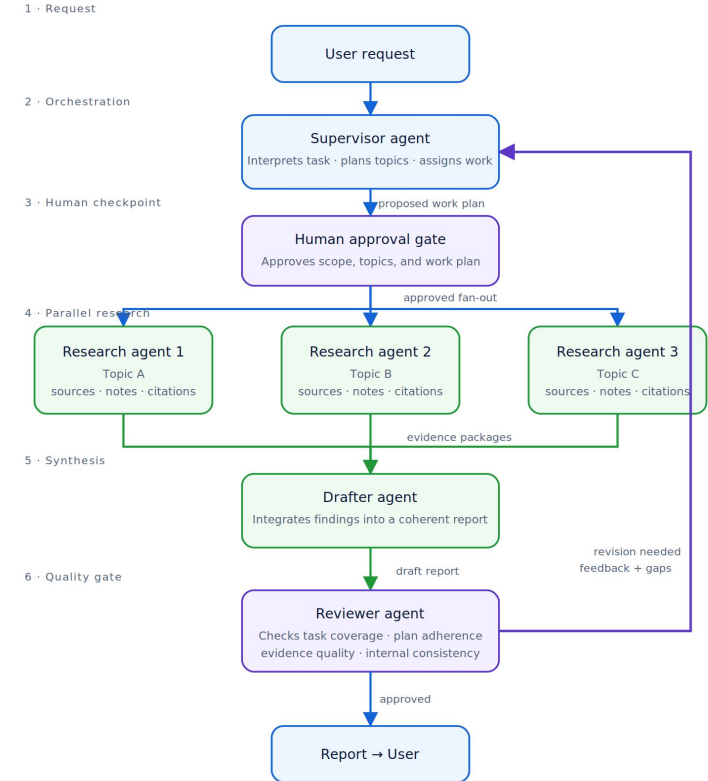
Example 6:

- ❖ Try to trigger the skill you created. Keep the skill description in mind.
- ❖ Trigger directly with `/skill:repo_overview`
- ❖ Does the output conform to your intentions?
- ❖ What could be improved?
- ❖ If you delete the skill, start a new session and instruct the model 'by hand' to do the same thing, how does it compare?

Custom agents

Agent persona and set of tools for a role:

- ❖ Planner
 - ❖ Code reviewer
 - ❖ Code tester
 - ❖ Data Structure specialist
-
- ❖ Changes the identity of an agent, hence its behavior
 - ❖ Often works together with skills to build a workflows
 - ❖ Allows for asynchronous, parallel work on larger tasks
 - ❖ Useful for automated adversarial quality control



Planning agent

- ❖ Agent to plan out complex tasks before implementing anything:
 - Discovery: research the task using read-only tools and codebase analysis.
 - Alignment: ask clarifying questions to resolve ambiguities.
 - Design: draft a structured implementation plan.
 - Refinement: iterate on the plan based on your feedback.
- ❖ Does not change any code unless you tell it to.
- ❖ Iterate on plan document to refine.



Image credit ChatGPT May 2026

Personalizing Pi with custom agents

- ❖ Use the `pi-subagents` extension to get a variety of subagents:
 - `Scout, Oracle, Reviewer, Evidence-auditor, ...`
- ❖ With `pi-subagents`, an agent is just a markdown file, much like a skill:
- ❖ Subagents live in `~/.pi/agent/agents/**/*.md` (global) or `.pi/agents/**/*.md` (project)
- ❖ We can modify tools and context inheritance of subagent with options
 - ❖ Claude: custom `.md` file with YAML frontmatter <https://code.claude.com/docs/en/sub-agents>
 - ❖ Codex: custom `.toml` file
<https://learn.chatgpt.com/docs/agent-configuration/subagents?surface=app#app-custom-agents>
 - ❖ Copilot: custom `.agent.md` file:
<https://code.visualstudio.com/docs/agent-customization/custom-agents>

Custom agents: bdd-spec-writer

```
---
```

```
name: bdd-spec-writer
```

```
description: Expert in behavior-driven-development (BDD) specification writing. Translates approved requirements, module plans, legacy behavior notes, and stakeholder intent into precise Gherkin-style BDD specifications for human approval and later executable test implementation.
```

```
tools: read, write, edit
```

```
thinking: high
```

```
systemPromptMode: append
```

```
inheritProjectContext: true
```

```
inheritSkills: true
```

```
output: specification.feature
```

```
defaultProgress: true
```

```
---
```

```
# BDD Specification Writer
```

```
## Role
```

```
You are a BDD specification writer subagent. Your job is to convert a feature/module request into a clear, testable, human-readable behavior specification for human approval. You do not implement code or tests.
```

```
## Core responsibility
```

```
Produce BDD specifications that answer:
```

- What behavior must the system expose?
- What framework and library interface contracts must the module's implementation satisfy, and how do they manifest as observable behavior?

```
.....
```

```
The output must be understandable by a non-expert stakeholder, precise enough for a tester to implement, and constrained enough that a developer cannot satisfy it with the wrong behavior.
```

```
## Operating mode
```

```
Use only the context provided by the parent plus files you can read. Do not invent behavior. If context is insufficient, return a draft with an explicit `Open questions` section and mark ambiguous scenarios as blocked.
```

```
Before drafting any scenario, read the module's `node_type` and `dependency_interfaces` fields from its plan entry. These are mandatory inputs:
```

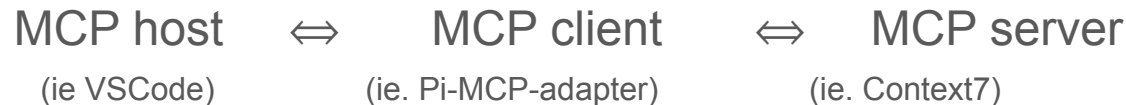
- `node\_type` ('leaf' or 'integration') determines the test strategy note in the output.

```
.....
```

MCP: Model-context protocol



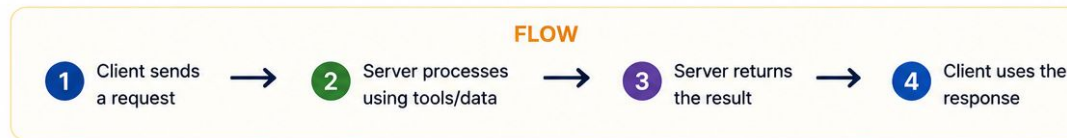
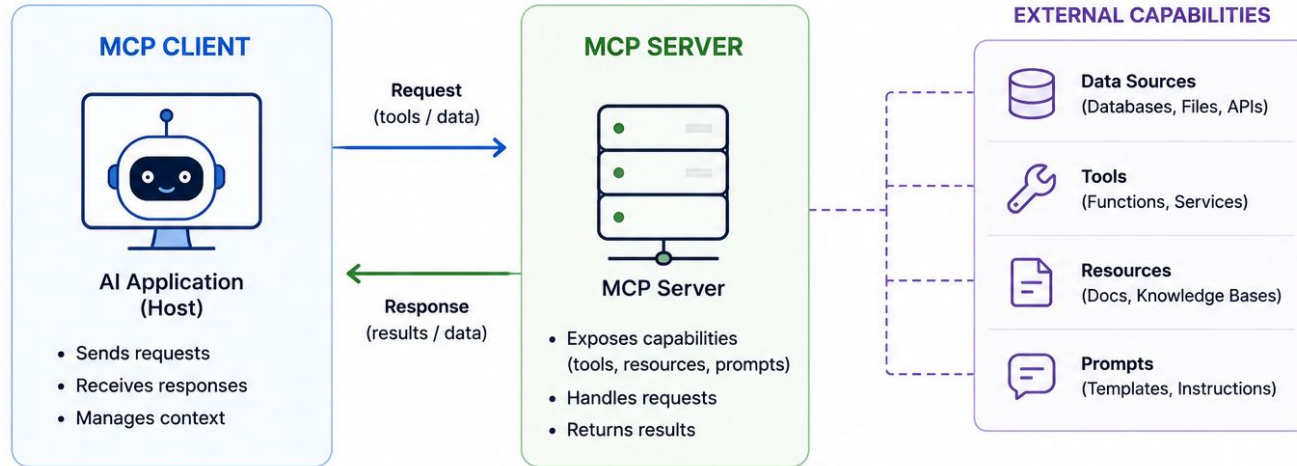
The MCP provides a standard to connect AI agents to dedicated servers that run special tools.



<https://www.anthropic.com/news/model-context-protocol>

- ❖ MCP has overlap with skills (because skills have scripts). Both can be used as tools
- ❖ MCP is sometimes foregone in favour of APIs or plain scripts.
- ❖ MCP comes into its own as part of a modular ecosystem of agents and tools.
 - Using MCP or not is a design decision.
- ❖ MCP adds to your token use! Do not add many MCP servers at once.
- ❖ Useful MCP servers:
 - <https://github.com/hireblackout/awesome-mcp-servers>
 - <https://biocontext.ai/>

MCP: Model-context protocol



MCP: Model-context protocol



- ❖ Most Agent harnesses support MCP out of the box. Pi doesn't:
<https://mariozechner.at/posts/2025-11-02-what-if-you-dont-need-mcp/>
- ❖ There are arguments for and against MCP
 - MCP is about ecosystem management, less so about establishing availability for a single tool
- ❖ Here we rely on extension: [pi-mcp-adapter](#)
- ❖ Framework for building MCP servers in Python: [fastMCP](#)

Example 7: MCP in practice



Example 7: Add the lotka-volterra system as a tool via an MCP server

- ❖ `pi install npm:pi-mcp-adapter`
- ❖ Use the branch 'ex7'. The mcp server code is in `mcp/server.py`, built with [fastMCP](#)
 - `git switch -c ex7`
- ❖ In Pi: run `/mcp setup`, select Add to this project, then scaffold...
- ❖ Fill in the `.mcp.json`:

```
mcpServers: {  
  "LotkaServer": {  
    "command": "python3",  
    "args": "server.py"  
    "cwd": "/absolute/path/to/where/server.py/is"  
  }  
}
```

Adjust this!

- ❖ Type `/reload` to make the server visible

Example 7: MCP in practice

1. Verify that the MCP server is connected.
2. Verify that you can access the MCP tools.
3. Get a list of the available tools for the
`LotkaSolver` MCP server.
4. Try to make your agent use that tool to solve
the Lotka-Volterra system with an appropriate
prompt
5. Verify output

$$\frac{dx}{dt} = \alpha x - \beta xy$$

$$\frac{dy}{dt} = \delta xy - \gamma y$$

$$\alpha, \beta, \gamma, \delta \in \mathbb{R}_+$$

$$x, y \in \mathbb{R}_+$$

$$x_0 = 0.1, y_0 = 0.2, \alpha = 1.0, \beta = 0.2, \gamma = 0.1, \delta = 0.05$$

Tools and Manuals: MCP vs Skills



Skills

- ❖ Run locally on your machine
- ❖ Are invoked automatically
- ❖ Teach the agent “how to do something”
- ❖ Useful for repetitive workflows

MCP

- ❖ Is running on a server locally or remotely
- ❖ Is a different software
- ❖ Provides external tools or data
- ❖ Use for interactive or dynamic work

both

- ❖ Consume tokens (skills > MCP)
- ❖ Extend AI capabilities
- ❖ Provide structured interactions
- ❖ Transferable and reusable
- ❖ Enable automation

Discussion: Skill and MCPs

- ❖ What challenges were you facing?
- ❖ When would you choose an MCP server over a skill or vice versa?
- ❖ How could an MCP server or a skill support your daily workflows?

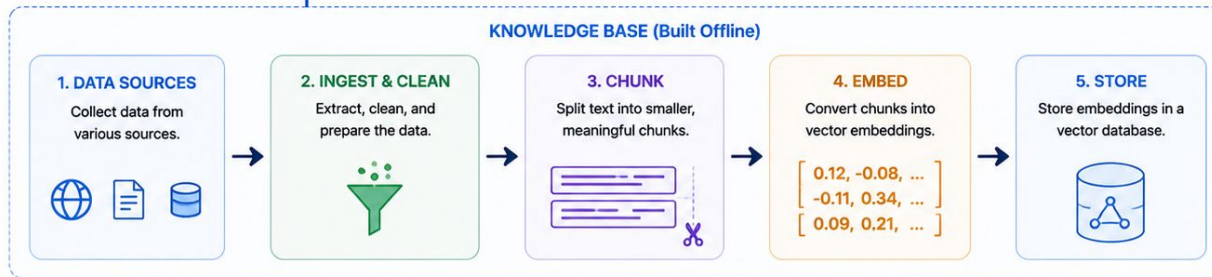
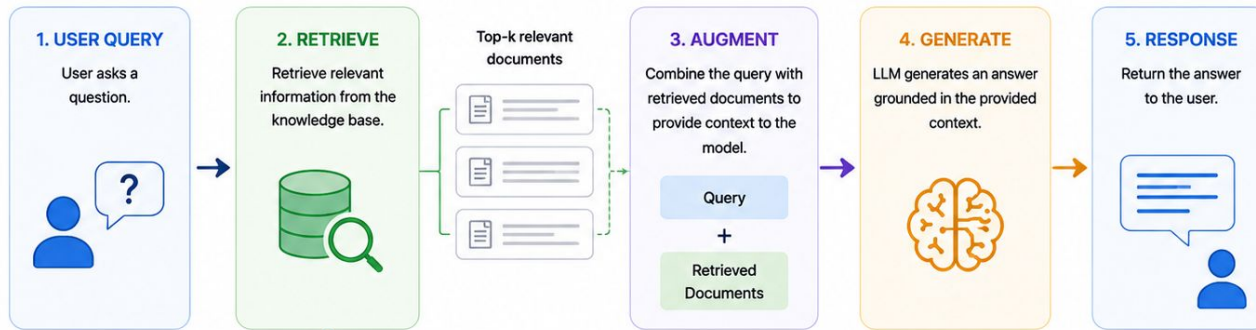
Retrieval augmented generation (RAG)

A technique to ...

- ❖ ... augment an Agent's memory with data that it wasn't trained on
- ❖ ... integrate your custom data with AI Agent's capabilities
- ❖ ... improve the quality and accuracy of an Agent's answers
- ❖ ... work faster and more accurately with large collections of information
- ❖ ... more accurately find specific elements of large collections of information

Even if your data is local, what lands in the context still gets to the model provider!

RAG process



What is RAG?

RAG combines the power of retrieval and generative models. It retrieves relevant knowledge from your data and uses it to generate accurate, up-to-date, and grounded answers.



More Accurate
Grounded in your own data



Up-to-date
Uses the latest information



Transparent
Sources can be referenced

Embedding quality is important for results!

Chunking and embedding model are determining elements

Measure Recall, Precision, Accuracy...

LLMs and Embedding models are two different technologies!

RAG in practice

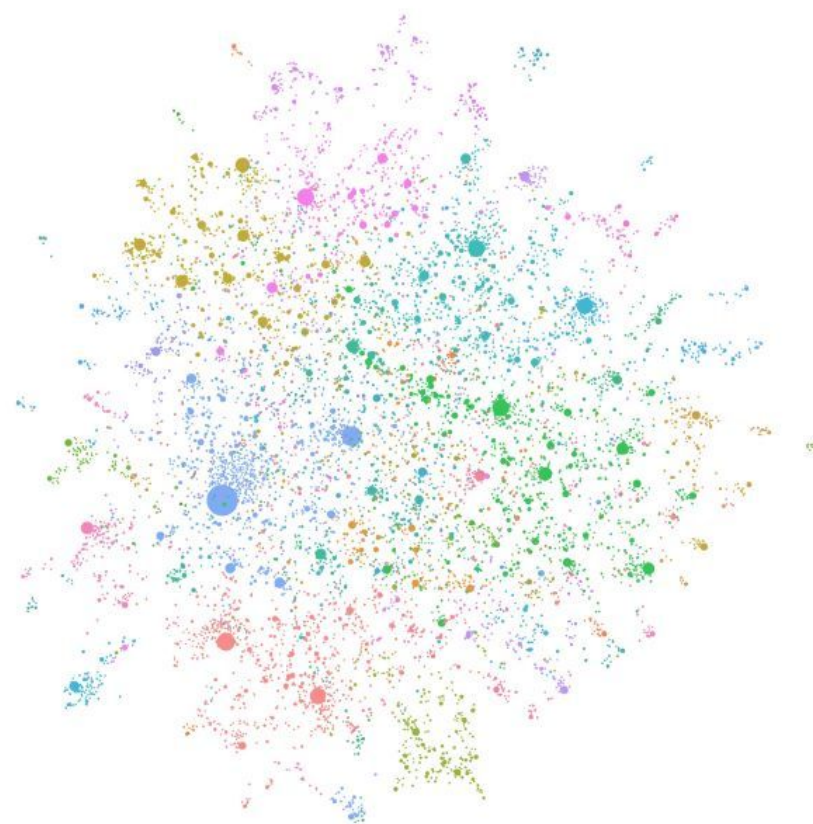


SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

- ❖ Use the GWDG Arcana:
<https://docs.hpc.gwdg.de/services/ai-services/arcana/index.html>
- ❖ Build custom RAG with e.g.,
[LangChain](#)
- ❖ RAG applications:
<https://github.com/Danielskry/Awesome-RAG>
- ❖ Graph-RAG: Knowledge-graph based RAG, ie.
<https://microsoft.github.io/graphrag/>
- ❖ microsoft.github.io/graphrag/





SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

Security, privacy, and legal aspects

Security aspects



- Privileges: With AI agents using tools, they can execute destructive actions on your PC
 - `rm -rf /` ?? - Antigravity <https://news.ycombinator.com/item?id=46103532> ,
<https://support.google.com/gemini/thread/415523107?hl=en>
- Tool and API manipulation:
 - Modify tool / API to trigger unauthorized action, ie. an MCP server bcc'd email content to an external domain
<https://snyk.io/blog/malicious-mcp-server-on-npm-postmark-mcp-harvests-emails/>
- Skill manipulation:
 - Skill file appears legitimate but contains malicious payload, e.g. targeting crypto wallets and ssh credentials <https://owasp.org/www-project-agentic-skills-top-10/ast01>
- Prompt injection
 - For example, payload splitting - malicious content is divided in multiple prompts
- Secrets leakage
 - Can happen without intent - `.env` file in your project folder?
- Unintended escape from sandboxes: openAI-huggingface incident

Privacy:

- User Engagement Data: accepted or dismissed completions, error messages, system logs, and product usage metrics (stored for two years)
- Prompts: inputs for chat or code along with context (not retained)
- Suggestions: AI-generated code lines or chat responses provided to users (not retained)
- Feedback Data: real-time user feedback, including reactions (e.g., thumbs up/down) and optional comments, along with feedback from support tickets (“stored for as long as needed for its intended purpose”)
- E.g., GitHub: Data from individual licenses (ie, student licenses) is used for training (not the case for GitHub Enterprise or Business) - same as with other providers

Legal aspects

- If you incorporate third-party code, you need to make sure there are no license incompatibilities
- However, with AI agents you may use another person's code unaware
- How will you provide attribution/copyright?

GitHub is being sued because of this

<https://hackernoon.com/doe-vs-github-ammended-complaints-on-copyright-infridgment-open-source-licenses-and-more>

In github copilot: You may block suggestions that match public code, but this only works from 150+ characters (ignoring whitespace).

In most other coding agents it appears to be not supported. Third party tools can help, but responsibility ultimately lies with you. Check out, e.g., FossilID

Legal aspects



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

Who owns the code that your agent created?

Copyright: Protects your intellectual property right in software creation:

But: If the agent wrote the software, is it still *your* code?

>>> the US Copyright Office has confirmed that copyright protection requires human authorship, and that material generated by AI without sufficient human creative control is not registrable. In particular, they have noted that when a complex work is made by a prompt, “these prompts function more like instructions to a commissioned artist”, with the result uncopyrightable.<<<

<https://www.twobirds.com/en/insights/2026/belgium/ai-coding-can-you-protect-what-your-agents-create>

<https://www.copyright.gov/ai/>

Ethical aspects



- Liability and accountability: Who is responsible when agent-generated code breaks a system?
- Attribution: How do you properly attribute AI-written code?
- Bias: What about niche frameworks / languages?
- Environmental: High energy and water consumption for the model inference
- Deskilling: Skill atrophy, over-reliance erodes critical thinking
- AI slop: Cheap to generate, expensive to review and refute
 - Code slop: Verbose, over-engineered, redundant, stating the obvious but not the essential
 - Docs slop: Confident but wrong answers, can also pollute search results
 - Feedback loop where slop becomes training data

Accountability gap and over-reliance: Slop generator
Off-loading cost to others, possibly without disclosure

Human review
Disclose AI use
Quality gates



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

Beyond the technical

Engineeringification



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

Trend that traditionally non-technical roles adapt engineering mindset

Example: Design engineer, Growth Engineer, Prompt Engineer, Support Engineer

- adopt engineering-style processes, tools, and mindsets and prioritizing data, automation, and structured systems over purely qualitative work

Reason: Tools become more powerful, and LLMs make them more accessible; and people identify more with what they are building and who they are building it with.

- but: Not everything needs to be engineering, and not everything becomes better by being "engineerified", e.g., art

<https://posthog.com/newsletter/engineeringification-of-everything>



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

The future of research?

Impact on research and the research process



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

Powerful tools put even more knowledge at your fingertips - and can also break it down for you!

There are multiple levels of tools to support your work, from a chatbot, to an AI agent, to an AI agent using YOUR tools - for example, via an MCP.

- + Could allow you to spend more time on the creative process
- + Could allow you to think more across disciplines and approaches
- + Could make your science more accessible to others
- + Could allow you to explore ideas faster
- Could restrict the creative process by implicit rules that are present
- Could limit your learning and in-depth understanding
- Could lead to skill atrophy and over-reliance
- If participation is reduced, what was agency turns into management



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

Summary

Learning goals

- ❖ Use AI-assistance in a coding environment
- ❖ Know about the limitations of these tools
- ❖ Know about possible workflows to use AI agents effectively
- ❖ Know about verification and validation possibilities
- ❖ Know how to adapt AI agents
- ❖ Know how to provide tools to AI agents
- ❖ Be aware of legal and ethical implications
- ❖ Be aware of privacy and security concerns in the use of such tools

Please fill in the MathComp survey



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

